Lesson Recapper

Learn on the fly

By Nico, Martha and Ben

What problem are we solving

People often miss meetings and lectures. The provided transcripts intended to follow catch up on the topic, are a terrible to learn from:

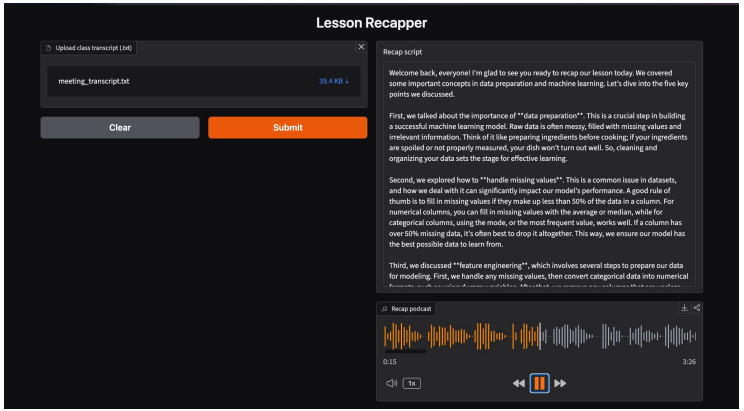
- **Raw transcripts are slow to read**: timestamps, filler words ("um", "uh") and tangents bury the key ideas.
- **Few students review them**, so the key ideas don't stick.
- **Reading is hard to fit** into a commute or a walk.

What is our solution

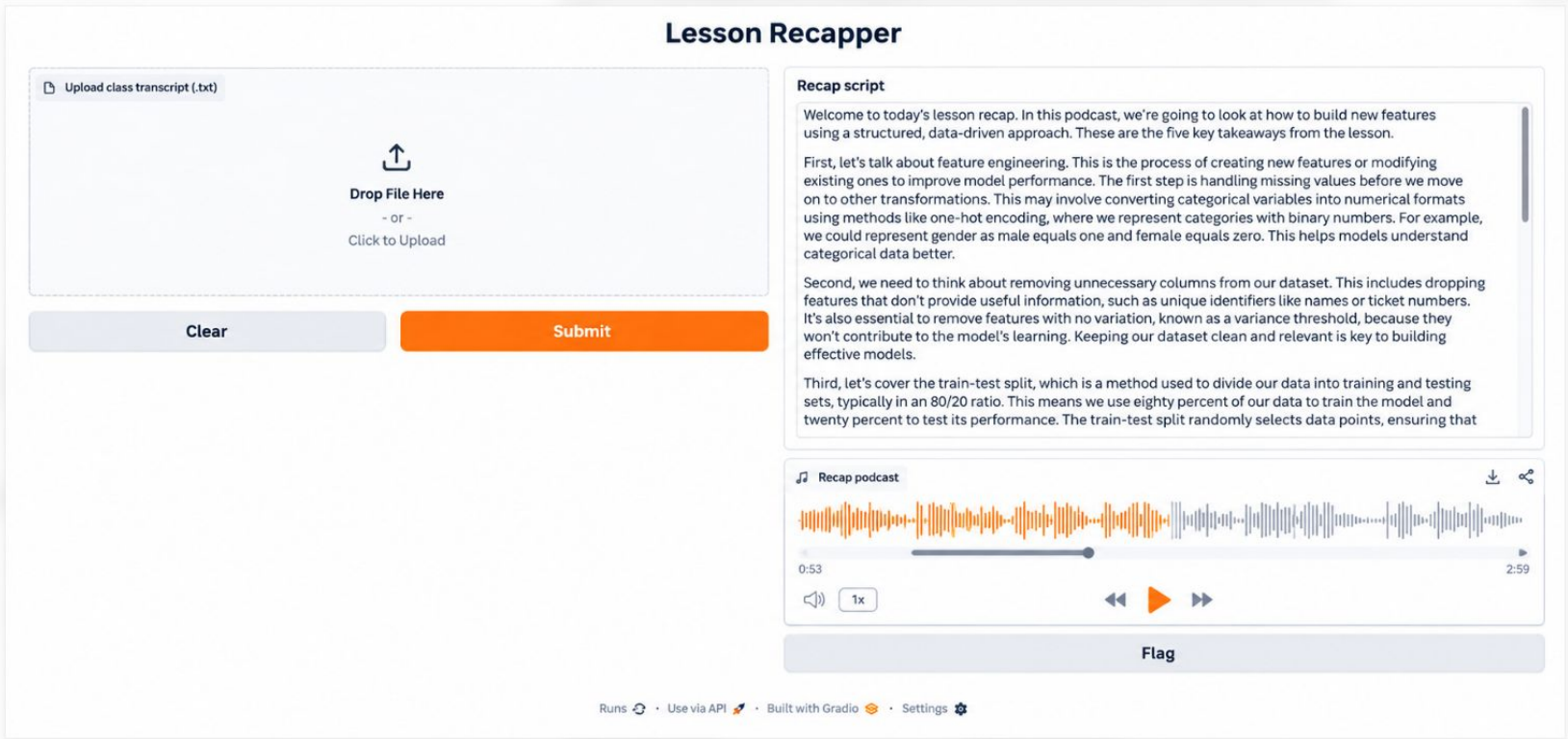
Listening to a summary is much easier. Which is why we built our Podcast Studio tool

You can:

- **Upload a transcript** on the web app
- **Get about 5 minutes of audio** from a friendly teaching assistant
- **Hear the 5 most important points** of the lesson
- **Review by listening**, on a commute or a walk



Demo Recapper Application



https://af67733044cb3ba150.gradio.live/

Key Challenges & Takeaways

Challenges

- **Git workflow mistakes:** When we didn't work on our own branch, forgot to pull before starting, or pushed code that wasn't ready, we got missing files, outdated code and merge conflicts that took time to fix.
- **Small details, big impact:** In VS Code, tiny, almost invisible mistakes made all the difference: a missing -r in pip install -r requirements.txt, VS Code running the code in the wrong environment, or a file name not passed to the script. Each one was easy to miss and enough to stop the whole program from running.

Takeaways

- **Branches protect main:** Each person works on their own branch, so main always stays a working version that everyone can rely on.
- **A clear Git routine:** Pull → work → commit → push → pull request. Following these steps every time prevents most conflicts.
- **One file, one job, one owner:** Splitting the project into separate .py files let us work in parallel. We only coordinated on main.py, the file everyone touches.
- **Agree on the hand-off:** Each step only needs to know what it receives and what it returns. For example, the data cleaning step passes one string of clean text to the LLM step.
- **Error messages are helpful:** The last line of the traceback usually says exactly what's wrong and how to fix it.
- **Start small, then improve:** Testing each module on its own before connecting them made problems much easier to find.